API_REFERENCE

LLMProvider Module - API Reference

Module: `digital.vasic.llmprovider` **Version:** 1.0.0 **Last Updated:** March 2026

Package `llmprovider`

The root package provides the core `LLMProvider` interface, circuit breaker, health monitor, retry logic, and lazy loading utilities for building fault-tolerant LLM provider integrations.

Interfaces

LLMProvider

The foundational interface that all LLM provider implementations must satisfy.

```
type LLMProvider interface {
    Complete(ctx context.Context, req *models.LLMRequest)
        (*models.LLMResponse, error)
    CompleteStream(ctx context.Context, req *models.LLMRequest)
        (<-chan *models.LLMResponse, error)
    HealthCheck() error
    GetCapabilities() *models.ProviderCapabilities
    ValidateConfig(config map[string]interface{}) (bool, []string)
}
```

Method	Description
`Complete`	Sends a synchronous completion request and returns a single response.
`CompleteStream`	

Method	Description
	Sends a streaming completion request and returns a channel of incremental responses. The channel is closed when the stream ends.
`HealthCheck`	Performs a health check against the provider. Returns `nil` if healthy.
`GetCapabilities`	Returns the provider's capabilities (streaming, tools, vision, concurrency limits, token limits).
`ValidateConfig`	Validates a configuration map. Returns `(true, nil)` if valid, or `(false, []string)` with validation error messages.

Circuit Breaker

Types

CircuitState

```go
type CircuitState string

const (
    CircuitClosed   CircuitState = "closed"
    CircuitOpen     CircuitState = "open"
    CircuitHalfOpen CircuitState = "half_open"
)
```

CircuitBreakerConfig

```go
type CircuitBreakerConfig struct {
    FailureThreshold    int           // Consecutive failures
        before opening (default: 5)
    SuccessThreshold    int           // Consecutive successes in
        half-open to close (default: 2)
    Timeout             time.Duration // Duration open before
        transitioning to half-open (default: 30s)
    HalfOpenMaxRequests int           // Max requests allowed in
        half-open state (default: 3)
}
```

CircuitBreakerStats

```go
type CircuitBreakerStats struct {
    ProviderID          string       `json:"provider_id"`
    State               CircuitState `json:"state"`
    TotalRequests       int64        `json:"total_requests"`
    TotalSuccesses      int64        `json:"total_successes"`
    TotalFailures       int64        `json:"total_failures"`
```

```
    ConsecutiveFailures  int
        `json:"consecutive_failures"`
    ConsecutiveSuccesses int
        `json:"consecutive_successes"`
    LastFailure          time.Time
        `json:"last_failure,omitempty"`
    LastStateChange      time.Time    `json:"last_state_change"`
}
```

CircuitBreakerListener

```
type CircuitBreakerListener func(providerID string, oldState,
        newState CircuitState)
```

Callback function invoked (in a goroutine with 5-second timeout) when the circuit state changes.

Constants

Constant	Value	Description
`MaxCircuitBreakerListeners`	100	Maximum number of listeners per circuit breaker to prevent memory leaks.

Sentinel Errors

Error	Description
`ErrCircuitOpen`	Returned when a request is rejected because the circuit is open.
`ErrCircuitHalfOpenRejected`	Returned when a request is rejected in half-open state because the max concurrent test requests limit is reached.

Constructor Functions

NewCircuitBreaker

```
func NewCircuitBreaker(providerID string, provider LLMProvider,
        config CircuitBreakerConfig) *CircuitBreaker
```

Creates a new circuit breaker wrapping the given provider with the specified configuration.

NewDefaultCircuitBreaker

```
func NewDefaultCircuitBreaker(providerID string, provider
        LLMProvider) *CircuitBreaker
```

Creates a circuit breaker with `DefaultCircuitBreakerConfig()` (5 failure threshold, 2 success threshold, 30s timeout, 3 half-open max requests).

DefaultCircuitBreakerConfig

```
func DefaultCircuitBreakerConfig() CircuitBreakerConfig
```

Returns sensible default values for circuit breaker configuration.

CircuitBreaker Methods

Method	Signature	Description
`Complete`	`(ctx, req) (*models.LLMResponse, error)`	Wraps provider `Complete` with circuit breaker logic.
`CompleteStream`	`(ctx, req) (<-chan *models.LLMResponse, error)`	Wraps provider `CompleteStream`. Stream success/failure tracked via wrapper channel.
`HealthCheck`	`() error`	Delegates directly to the underlying provider (no circuit logic).
`GetCapabilities`	`() *models.ProviderCapabilities`	Delegates directly to the underlying provider.
`ValidateConfig`	`(config) (bool, []string)`	Delegates directly to the underlying provider.
`GetState`	`() CircuitState`	Returns the current circuit state (thread-safe).
`GetStats`	`() CircuitBreakerStats`	Returns a snapshot of circuit breaker statistics.

Method	Signature	Description
`Reset`	`()`	Resets the circuit breaker to closed state and clears all counters.
`IsOpen`	`() bool`	Returns true if the circuit is in the open state.
`IsClosed`	`() bool`	Returns true if the circuit is in the closed state.
`IsHalfOpen`	`() bool`	Returns true if the circuit is in the half-open state.
`AddListener`	`(CircuitBreakerListener) int`	Registers a state change listener. Returns listener ID, or -1 if max listeners reached.
`RemoveListener`	`(id int) bool`	Removes a listener by ID. Returns true if found and removed.
`ListenerCount`	`() int`	Returns the current number of registered listeners.

CircuitBreakerManager

Manages multiple circuit breakers for a fleet of providers.

Constructor Functions

```
func NewCircuitBreakerManager(config CircuitBreakerConfig)
        *CircuitBreakerManager
func NewDefaultCircuitBreakerManager() *CircuitBreakerManager
```

Methods

Method	Signature	Description
`Register`	`(providerID string, provider LLMProvider) *CircuitBreaker`	Registers a provider and returns its circuit breaker.
`Get`	`(providerID string) (*CircuitBreaker, bool)`	Retrieves the circuit breaker for a provider.
`Unregister`	`(providerID string)`	Removes a provider's circuit breaker.
`GetAllStats`	`() map[string]CircuitBreakerStats`	Returns statistics for all registered circuit breakers.
`GetAvailableProviders`	`() []string`	Returns IDs of providers with closed or half-open circuits.
`ResetAll`	`()`	Resets all circuit breakers to closed state.

Health Monitor

Types

HealthStatus

```go
type HealthStatus string

const (
    HealthStatusHealthy   HealthStatus = "healthy"
    HealthStatusUnhealthy HealthStatus = "unhealthy"
    HealthStatusUnknown   HealthStatus = "unknown"
    HealthStatusDegraded  HealthStatus = "degraded"
)
```

HealthMonitorConfig

```go
type HealthMonitorConfig struct {
    CheckInterval      time.Duration // How often to run checks
        (default: 30s)
    HealthyThreshold   int           // Consecutive successes to
        mark healthy (default: 2)
    UnhealthyThreshold int           // Consecutive failures to
        mark unhealthy (default: 3)
    Timeout            time.Duration // Per-check timeout
        (default: 10s)
    Enabled            bool          // Whether monitoring is
        enabled (default: true)
}
```

ProviderHealth

```go
type ProviderHealth struct {
    ProviderID       string        `json:"provider_id"`
    Status           HealthStatus  `json:"status"`
    LastCheck        time.Time     `json:"last_check"`
    LastSuccess      time.Time     `json:"last_success,omitempty"`
    LastError        string        `json:"last_error,omitempty"`
    ConsecutiveFails int           `json:"consecutive_fails"`
    Latency          time.Duration `json:"latency,omitempty"`
    CheckCount       int64         `json:"check_count"`
    SuccessCount     int64         `json:"success_count"`
    FailureCount     int64         `json:"failure_count"`
}
```

AggregateHealth

```go
type AggregateHealth struct {
    OverallStatus      HealthStatus
        `json:"overall_status"`
```

```
    TotalProviders     int
        `json:"total_providers"`
    HealthyProviders   int
        `json:"healthy_providers"`
    DegradedProviders  int
        `json:"degraded_providers"`
    UnhealthyProviders int
        `json:"unhealthy_providers"`
    UnknownProviders   int
        `json:"unknown_providers"`
    Providers          map[string]HealthStatus `json:"providers"`
}
```

HealthListener

```
type HealthListener func(providerID string, oldStatus, newStatus
        HealthStatus)
```

Constructor Functions

```
func NewHealthMonitor(config HealthMonitorConfig) *HealthMonitor
func NewDefaultHealthMonitor() *HealthMonitor
func DefaultHealthMonitorConfig() HealthMonitorConfig
```

HealthMonitor Methods

Method	Signature	Description
`RegisterProvider`	`(providerID string, provider LLMProvider)`	Registers a provider for health monitoring. Initial status is `unknown`.
`UnregisterProvider`	`(providerID string)`	Removes a provider from monitoring.
`AddListener`	`(HealthListener)`	Adds a listener for health status changes.
`Start`	`()`	Begins the periodic health monitoring loop. Runs an initial check immediately.
`Stop`	`()`	

Method	Signature	Description
		Stops the health monitoring loop.
`IsRunning`	`() bool`	Returns true if the monitor is actively running.
`GetHealth`	`(providerID string) (*ProviderHealth, bool)`	Returns health status for a specific provider (returns a copy).
`GetAllHealth`	`() map[string]*ProviderHealth`	Returns health status for all providers.
`GetHealthyProviders`	`() []string`	Returns IDs of all healthy providers.
`IsHealthy`	`(providerID string) bool`	Returns true if the specified provider is healthy.
`RecordSuccess`	`(providerID string)`	Manually records a successful operation.
`RecordFailure`	`(providerID string, err error)`	Manually records a failed operation.
`GetAggregateHealth`	`() AggregateHealth`	Returns overall system health summary.
`ForceCheck`	`(providerID string) error`	Forces an immediate health check for a specific provider.
`GetConfig`	`() HealthMonitorConfig`	Returns the current monitor configuration.

Retry Logic

Types

RetryConfig

```
type RetryConfig struct {
    MaxRetries   int           // Maximum retry attempts, 0 = no
        retries (default: 3)
    InitialDelay time.Duration // Initial backoff delay (default:
        1s)
    MaxDelay     time.Duration // Maximum backoff delay (default:
        30s)
    Multiplier   float64       // Exponential backoff multiplier
        (default: 2.0)
    JitterFactor float64       // Jitter factor 0.0-1.0 (default:
        0.1)
}
```

RetryResult

```
type RetryResult struct {
    Response   *http.Response
    Attempts   int
    LastError  error
    TotalDelay time.Duration
}
```

RetryableFunc

```
type RetryableFunc func() (*http.Response, error)
```

Functions

DefaultRetryConfig

```
func DefaultRetryConfig() RetryConfig
```

Returns sensible defaults (3 retries, 1s initial delay, 30s max, 2.0 multiplier, 0.1 jitter).

IsRetryableStatusCode

```
func IsRetryableStatusCode(statusCode int) bool
```

Returns true for HTTP status codes 429, 500, 502, 503, 504.

IsRetryableError

```
func IsRetryableError(err error) bool
```

Returns true for retryable errors (network errors). Returns false for `context.Canceled` and `context.DeadlineExceeded`.

ExecuteWithRetry

```
func ExecuteWithRetry(ctx context.Context, config RetryConfig, fn
        RetryableFunc) (*RetryResult, error)
```

Executes a function with retry logic and exponential backoff with jitter. Respects context cancellation. Closes response bodies before retrying on retryable HTTP status codes.

CalculateBackoff

```
func CalculateBackoff(attempt int, config RetryConfig)
        time.Duration
```

Calculates the backoff duration for a given attempt number using exponential backoff with jitter.

RetryableHTTPClient

Wraps `http.Client` with automatic retry logic.

```
func NewRetryableHTTPClient(client *http.Client, config
        RetryConfig) *RetryableHTTPClient
```

Method	Signature	Description
`Do`	`(ctx context.Context, req *http.Request) (*http.Response, error)`	Executes an HTTP request with retry logic. Clones the request for each attempt.
`GetConfig`	`() RetryConfig`	Returns the retry configuration.

Thread Safety

All exported types and methods are safe for concurrent use:

- `CircuitBreaker` and `CircuitBreakerManager` use `sync.RWMutex` for state management.
- `HealthMonitor` uses `sync.RWMutex` with per-provider goroutines for concurrent health checks.
- `RetryConfig` is effectively immutable after creation.

- Listener notifications are dispatched in separate goroutines with 5-second timeouts to prevent blocking.

Dependencies

Dependency	Purpose
`digital.vasic.models`	`LLMRequest`, `LLMResponse`, `ProviderCapabilities` types
`github.com/sirupsen/logrus`	Structured logging in circuit breaker
Go standard library	`context`, `sync`, `time`, `net/http`, `math`, `math/rand`

NVIDIA Nemotron RAG Support

HelixAgent provides comprehensive integration with NVIDIA's Nemotron RAG models for document processing, multimodal understanding, and grounded answer generation.

Supported Models

Model	Type	Purpose	Context Window
`nvidia/llama-nemotron-embed-vl-1b-v2`	Embedding	Multimodal document embedding (text + images)	2048-dim vectors
`nvidia/llama-nemotron-rerank-vl-1b-v2`	Reranking	Cross-encoder relevance scoring with vision	VLM-based
`nvidia/llama-3.3-nemotron-super-49b-v1.5`	Generation	Citation-backed answer generation	128K tokens
`nvidia/nemotron-ocr`	Extraction	Document OCR and structure extraction	N/A

Provider Configuration

```
nemotronProvider := &NemotronProvider{
    BaseURL:     "https://integrate.api.nvidia.com/v1",
    APIKey:      os.Getenv("NVIDIA_API_KEY"),

    // Model endpoints
```

```
    EmbedModel:   "nvidia/llama-nemotron-embed-vl-1b-v2",
    RerankModel:  "nvidia/llama-nemotron-rerank-vl-1b-v2",
    GenerateModel: "nvidia/llama-3.3-nemotron-super-49b-v1.5",

    // Processing options
    ChunkSize:      512,
    ChunkOverlap:   100,
    ExtractTables:  true,
    ExtractCharts:  true,
    TableFormat:    "markdown",
    RequireCitations: true,
}

// Register with health monitoring
monitor.RegisterProvider("nvidia-nemotron", nemotronProvider)
```

RAG Query with Citations

```
result, err := nemotronProvider.RAGQuery(ctx, &RAGQueryRequest{
    Document: "financial_report.pdf",
    Query: "What was Q3 revenue growth?",
    Options: &RAGOptions{
        TopK: 10,
        RerankTopK: 5,
        CitationLevel: CitationLevelPageSection,
        IncludeSourceText: true,
    },
})

// Result includes:
// - Answer with inline citations
// - Source references (page, section)
// - Confidence score
// - Extracted context
```

Key Features

- **Multimodal Understanding:** Process charts, diagrams, and tables alongside text
- **Structured Extraction:** Preserve table relationships with Markdown output
- **Citation-Backed Answers:** Every claim traceable to source document
- **Vision-Language Models:** Understand visual content in documents
- **Enterprise-Grade:** Supports compliance and audit requirements

System Requirements

- **GPU:** Minimum 24GB VRAM for local model deployment
- **Storage:** 250GB for models, datasets, and vector database
- **Python:** 3.10-3.12 (for NeMo Retriever library)
- **API Key:** Free access at build.nvidia.com

Documentation

For complete integration guide, see:

- NVIDIA Nemotron RAG Integration Guide
- NVIDIA Developer Blog: Document Processing Pipeline
- NeMo Retriever Library

Last Updated: April 2026